

dr. Matej Šprogar

Podprogrami

Ali zakaj in kako organizirati programsko kodo

Deli-in-vladaj s podprogrami

Največji problem programiranja je **kompleksnost** rešitev, ki jo zmoremo obvladati le s pristopom **deli-in-vladaj** (*divide et impera*). Ideja je, da velik problem razgrajujemo na manjše podprobleme, dokler ne pridemo do stopnje, ki jo *znamo* sprogramirati.

Obstaja še kopica drugih principov, a noben ni tako **univerzalen** in **enostaven** (to sta kritični lastnosti, ki ju premorejo le *najboljši* algoritmi!).

Prva uporaba principa *deli-in-vladaj* pogosto projekt razgradi na tri tipične faze:

1. priprava podatkov
2. računanje
3. prikaz/uporaba rezultatov

Potem ponovimo *deli-in-vladaj* še na vsaki fazi...

Dejansko je princip uporaben tako za **vsebino** kot tudi za **vodenje** projekta!

Nazaj k 42...

Preprost prikaz odgovora na vprašanje o življenju, vesolju in sploh vsem smo nekoč že samo-intuitivno razgradili na dve fazi: izračun in prikaz odgovora.

Prikaza nismo izvedli sami, ampak smo ga prepustili vgrajenemu sistemskemu podprogramu, ki izpiše **našo** vsebino (42) v okviru konzolnega okna.

Náša koda ni poskrbela ne za konzolno okno, ne za ozadje, menuje, postavitev, fonte, barve, miško... Raje smo **uporabili** znanje, ki je bilo skrito v temu namenjenem podprogramu.

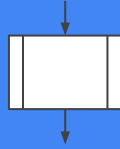
Preprost ukaz (`System.Console.WriteLine()` (C#) oz. `System.out.println()` (Java)) sproži sicer kompliciran prikaz rezultata na monitorju.

Ukaz *skriva* na stotine ukazov, ki zahtevajo obsežno sistemsko znanje in bi jih sicer morali pisati sami; sedaj jih lahko "zastonj" uporabimo znova in znova:

```
System.Console.WriteLine(6*7);  
System.Console.WriteLine(21*2);  
System.Console.WriteLine(42);
```

bo izpisalo:

424242



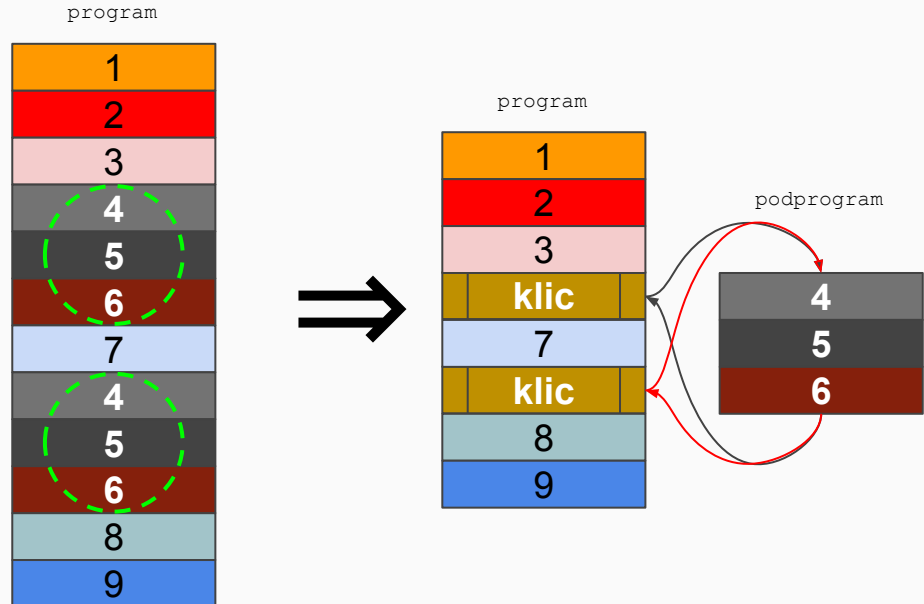
Podprogram

Podprogram je košček **kode**, ki jo lahko *zaženemo* (temu rečemo tudi *klic* podprograma), ko želimo.

Podprogrami omogočajo tako **razgradnjo** kot tudi **ponovno uporabo** kode - z njimi ubijemo *dve muhi na en mah!*

S podprogramom lahko dolgo kodo **vedno** razbijemo na **dva** kosa; tipično "izluščimo" kodo, ki se ponavlja in je logično celota. Dobro imenovan podprogram bo tako razgrajeno kodo naredil tudi mnogo bolj berljivo in s tem človeku razumljivo (prevajalniku je itak vseeno).

Isti podprogram lahko izvedemo **večkrat** in to iz **različnih točk** v kodi. Po zaključku podprograma se bo izvajanje našega programa nadaljevalo, kot da ga nismo nikoli zapustili.



Hierarhija podprogramov

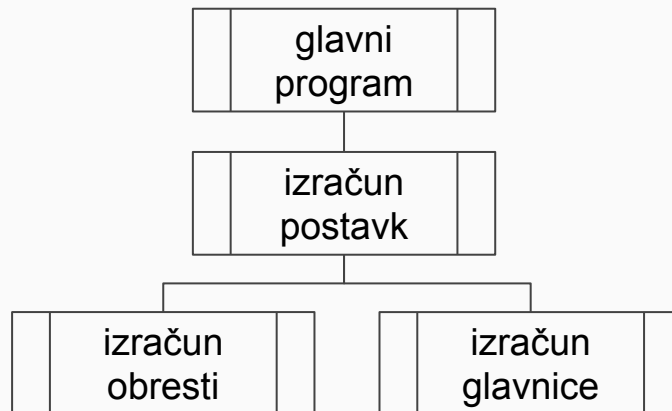
Glavni podprogram (*main*) je zgolj tisti izmed vseh podprogramov, ki se prične izvajati ob zagonu programa (klican s strani OS), ostali pa čakajo na svoj klic.

Hierarhično gledano je *glavni* program na najvišjem, ničtem nivoju; podprogrami klicani iz njega so nivo nižje na *globini* 1, klici od tam gredo v globino 2 in tako naprej.

Kodo, iz katere kličemo podprogram, imenujemo *klicajoča* koda, sam podprogram pa je *klicana* koda. Npr. *izračun_postavk* je klican iz *glavnega* programa.

Podprogrami so *neodvisni* bloki kode: njihove spremenljivke so *lokalne* narave in jih ostali podprogrami zato **ne vidijo** (super varno!).

Program lahko organiziramo kot množico podprogramov, ki so v nekakšni hierarhiji glede na to, kdo koga kliče.



Definicija

Podobno, kot moramo definirati spremenljivke, moramo definirati tudi podprograme.

C++, Java in C# podprogram mora imeti znane najmanj **ime**, **tip** in **telo**. *Ime* potrebujemo za proženje ukazov v njegovem *telesu* (bloku kode), *tip* pa se nanaša na povratno* informacijo.

Okrogli oklepaji za imenom pomagajo razlikovati **klic** podprograma od uporabe imena podprograma.

C++ in Java zahtevata **definicijo** podprogramov **izven** **teles** podprogramov, C# pa omogoča tudi *lokalne* podprograme. Splošna definicija podprograma brez vhodnih argumentov je:

```
tip ime_podprograma()  
{  
    telo_podprograma;  
}
```

Podprogram **kličemo** (sprožimo) z imenom in **oklepaji**:

```
ime_podprograma ( ) ;
```

Klic podprograma je mogoč samo **znotraj** **telesa** (klicajočega) podprograma oziroma na mestih*, kjer potrebujemo rezultat.

Povratna informacija

Podprogram v C++, Javi in C# lahko klicajoči kode **vrne** *eno** vrednost določenega tipa s pomočjo rezervirane besede **return**, ki podano mu vsebino posreduje na mesto klica funkcije na višjem nivoju.

Ukaz *return* **zaključi** izvajanje podprograma in vrne nadzor klicajočemu podprogramu na višjem nivoju. Podprogram lahko vsebuje več ukazov *return*, vsak od njih zaključi eno izvajalno pot (dva *return*a v sekvenci nimata smisla).

Podprogram lahko ob zaključku klicajoči kodi vrne *eno** vrednost, ki "kot da zasede" mesto funkcijskega klica:

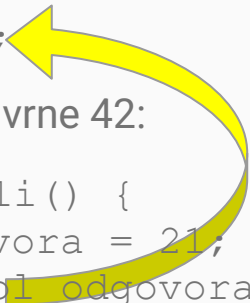
```
int odgovor = mocno_premisli();
```

postane

```
int odgovor = 42;
```

če le podprogram dejansko vrne 42:

```
int mocno_premisli() {  
    int pol_odgovora = 21;  
    return 2 * pol_odgovora;  
}
```



Klicatelj lahko vrnjeno vrednost tudi ignorira (je ne uporabi ali shrani):

```
mocno_premisli();
```

Učinek *returna*

Klic podprograma je po učinku enak, kot če bi se podprogramovo telo izvedlo v okviru klicajoče kode, nato pa bi se nek odgovor, ki ga je proizvedel podprogram, shranil v spremenljivki ali uporabil v izrazu.

Če želi funkcija vrniti več vrednosti, jih mora ali "zapakirati" v en return paket, ali (C# in C++) uporabiti klic argumentov po referenci namesto kopiji vrednosti (o tem kasneje).

```
tip ime_podprograma() {  
    telo_podprograma;  
    return odgovor_podprograma;  
}
```

```
tip rezultat = ime_podprograma();
```

je enakovredno

```
tip rezultat;  
{  
    telo_podprograma;  
    rezultat = odgovor_podprograma;  
}
```


Vhodni argumenti

Podprogrami tipično potrebujejo informacije iz *zunanjega* (klicajočega) sveta - nekako moramo podprogramu podati vse podatke, ki jih potrebuje za svoje delo. Temu služijo **argumenti**, ki jih ob *klicu* podprograma enostavno naštejemo med okroglimi oklepaji.

Parametrom, ki jih navedemo ob *klicu* podprograma, pravimo **dejanski argumenti**, saj že nosijo neko vrednost. **Formalni argumenti** v *definiciji* pa ustvarijo prostor, kamor se bodo skopirale vrednosti dejanskih argumentov ob klicu.

Argumenti so lokalne spremenljivke v okviru podprograma in so **kopija** podatkov, ki so bili podani ob klicu podprograma:

```
int osnova = 40;  
osnova = pristej2(osnova);
```



Možna definicija podprograma *pristej2* je:

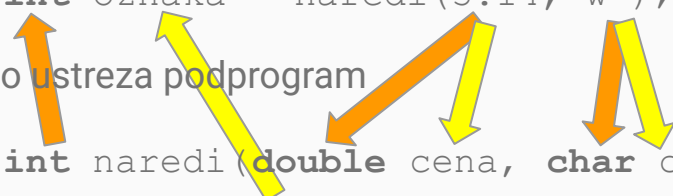
```
int pristej2(int temelj) {  
    return temelj + 2;  
}
```

Podprogram *pristej2* ne more uporabiti (brati) spremenljivke *osnova* iz podprograma na višjem nivoju, je pa zato *temelj* enakovreden podatek!

Če imamo več vhodnih argumentov

Vrstni red in tipi argumentov so pomembni - **vrednosti** dejanskih argumentov morajo po številu in **tipu** ustrezati formalnim argumentom v definiciji podprograma. Klicu

`int oznaka = naredi(3.14, 'w');`
recimo ustreza podprogram
`int naredi(double cena, char c)`
`{ ... return id; }`



C# pa pri klicu omogoča tudi imenovane argumente (*named arguments*) – namesto s pozicijo (prvi, drugi... argument) lahko ciljno spremenljivko izberemo z njenim imenom:

`naredi(c:'w', cena:3.14);`
je enakovredno
`naredi(cena:3.14, c:'w');`
je enakovredno
`naredi(3.14, 'w');`

Privzeti argumenti

Klic funkcije vedno zahteva vse argumente. Lepo bi bilo, če bi morali ponavljajoče se argumente podati le, če bi odstopali od neke "pričakovane" vrednosti.

Recimo funkcija za oblikovanje števila pričakuje tudi številsko osnovo, v kateri naj oblikuje število:

```
string oblikuj(int stevilo,  
               int osnova) { ... }
```

```
oblikuj(42, 10);    // "42"  
oblikuj(42, 2);     // "101010"
```

Ker pa jo večinoma uporabljamo za oblikovanje desetiških števil, lahko definicijo procedure dopolnimo in argumentu `osnova` določimo **privzeto** vrednost:

```
string oblikuj(int stevilo,  
               int osnova = 10) { ... }
```

Ta definicija omogoča dva enakovredna ukaza:

```
oblikuj(42);        // "42"  
oblikuj(42, 10);    // "42"
```

Argumenti po vrednosti in referenci

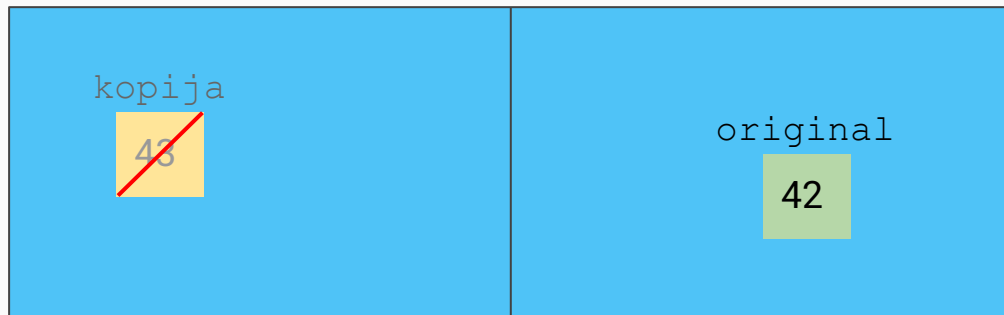
Argument je **dodatna** spremenljivka, ki je dostopna samo podprogramu. Spreminjanje *kopije* argumenta v podprogramu zato **ne vpliva** na vrednost originala.

Če bi funkcija vseeno hotela vplivati na original v klicajoči rutini, bi morala poznati njegovo lokacijo v pomnilniku. Java tega ne omogoča, C++ in C# pa s posebnim mehanizmom *referenc**.

* beseda *referenca* v Javi, C# in C++ ne pomeni isto, čeprav ima podobno uporabo. Več o tem drugje.

```
void povecaj_in_izpisi(int kopija) {  
    kopija += 1;  
    System.out.print(kopija);  
}
```

```
int original = 42;  
povecaj_in_izpisi(original); // 43  
System.out.print(original); // 42
```



Prenos argumentov po referenci (C++, C#)

Navaden argument omogoča *enosmeren* prenos vrednosti (kopija iz klicajoče v klicano funkcijo), za obratno smer imamo *return*. Ker pa slednji vrne največ eno* vrednost, lahko v C++ in C# spremenimo "naravo" argumenta funkcije.

Namesto, da bi bil kopija originala, lahko postane **drugo ime** za original.

```
void fun(ref int drugo_ime_za_original) { ... } // C#  
void fun(int& drugo_ime_za_original) { ... }    // C++
```

Za "zapleteno" mehaniko s kazalci v ozadju poskrbi prevajalnik (fino!).

Referenčni argument C#

S takšnim argumentom lahko funkcija "nehote" poseže v podatke klicajoče funkcije, kar je sicer nemogoče in ravno zato tudi nevarno.

C# zato od programerja zahteva, da pri vsakem klicu takšne funkcije to tudi eksplicitno dovoli, ker to pomeni, da je programer seznanjen z morebitno spremembo svojega podatka *mimo return* mehanizma...

C# pozna tudi **out** za izključno izhodne argumente, ki celo bolj ustreza primeru na desni...

```
void popravi(ref int karkoli) {  
    karkoli = 42;  
}
```

Klic funkcije je v C# očitno drugačen:

```
int odgovor = 0;  
popravi(ref odgovor);
```

odgovor
karkoli

42

V takšno funkcijo ne moremo poslati nič, česar ne moremo spremeniti (npr. konstant, izrazov):

```
popravi(ref 3); // NAPAKA  
popravi(ref odgovor+1); // NAPAKA
```

Kaj pa Java? (velja tudi za C#)

Java nima prenosa argumentov po referenci, čeprav pozna reference.

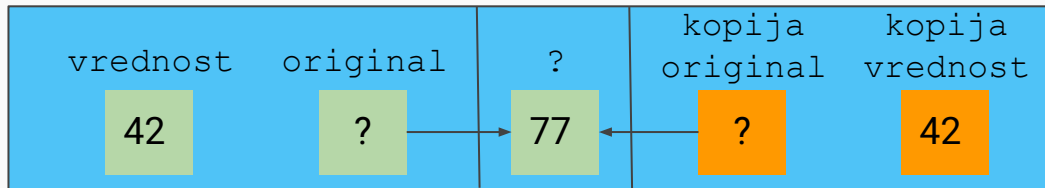
Referenčni argument je specifika, ki omogoča spremembo spremenljivke *vrednostnega* tipa tudi v podprogramu. V Javi pa je večina podatkov referenčnega tipa, zato referenčni argumenti niso nujnost.

Uporaba reference je določena s tipom podatka. Vsi *objekti* so referenčne spremenljivke, medtem ko (nam znani) osnovni tipi *NISO reference*, ampak *vrednosti*. Več o tem drugje.

V **Javi** in **C#** je referenca spremenljivka, ki hrani programerju neznan naslov, *obnaša pa se kot vsebina na tem naslovu*.

Ker je funkcijski argument kopija reference, lahko spremenimo vsebino, ki jo originalna referenca naslavlja, ne pa tudi same reference.

```
int vrednost = 42;  
Stevilo original = new Stevilo(77);  
podprogram(vrednost, original);
```



Procedure in funkcije

Naloga podprograma je, da na ukaz opravi neko nalogo. Če zgolj nekaj opravi, ga imenujemo **procedura**, če pa (dodatno) klicajoči kodi *vrne* še nek rezultat, ga imenujemo **funkcija**.

Vsaka funkcija lahko opravi delo procedure, ki pa ne zmore vrniti rezultata. Funkcija je torej splošnejši koncept (v toliko ni več potrebe, da sploh še govorimo o procedurah, čeprav je razlikovanje včasih koristno).

Funkciji v okviru razreda* (*class*) pravimo tudi **metoda**.

Procedura ima **stranski** učinek glede na klicajočo kodo (zato je ime procedure tako zelo pomembno!):

```
prizgi_luc();
```

Funkcija *zmore* enako, **dodatno** pa nam *nazaj* sporoči še neko informacijo/rezultat:

```
bool svetlo = prizgi_luc();
```

Proceduro definiramo kot funkcijo *brez* povratne informacije (*void*).

```
void moja_procedura() {...}  
tip moja_funkcija() {...}
```


Funkcija ali procedura?

Če želi procedura doseči trajni učinek, mora spremeniti nekaj *zunanjega* (njeni podatki izginejo ob zaključku). Je dobra za opis postopka rešitve.

Idealna funkcija nasprotno ne spremeni nič - trajni učinek bi naj ustvarila šele klicajoča koda na podlagi vrnjenega rezultata. Klicajoča koda je potem sicer daljša, je pa zato program bolj **razumljiv** in **prilagodljiv**!

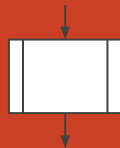
V praksi se funkcija in procedura prepletata in tudi C++, Java in C# ločujejo med njima zgolj na podlagi tega, ali podprogram kaj vrača.

Na primer funkcija *vecje*(*a*, *b*) vrne zgolj večjo od dveh vrednosti, brez stranskih učinkov (recimo izpisa na ekran). *Morebiten* izpis na ekran mora zato ustrezno opraviti klicajoča koda:

```
int maks = vecje(42, 7);  
System.Console.Write(maks);
```

Nasprotno bi procedura *izpisi_vecje* najavila stranski učinek izpisa v imenu procedure (kam točno se bo sploh izpisala večja vrednost?).

Procedura *izpisi_vecje* je slabše *ponovno uporabna* 😞 (če hočemo recimo izpis na tiskalnik...) kot funkcija *vecje*, ki jo lažje *kombiniramo*.



Podprogram

Podprogram (tudi funkcija, procedura, metoda, (sub)rutina...) je blok ukazov, ki ga kličemo po potrebi.

Vsak podprogram ima določen nabor argumentov (parametrov), ki so **kopije*** podatkov iz klicajočega podprograma.

Uporabljamo jih za

1. razgradnjo problema
2. ponovno uporabo kode

Za doseganje obeh ciljev je kritično dobro poimenovanje!

funkcija[#] moj_podprogram(argumenti...)

...

[**vrni** povratna_informacija]
konec funkcije

kliči moj_podprogram()

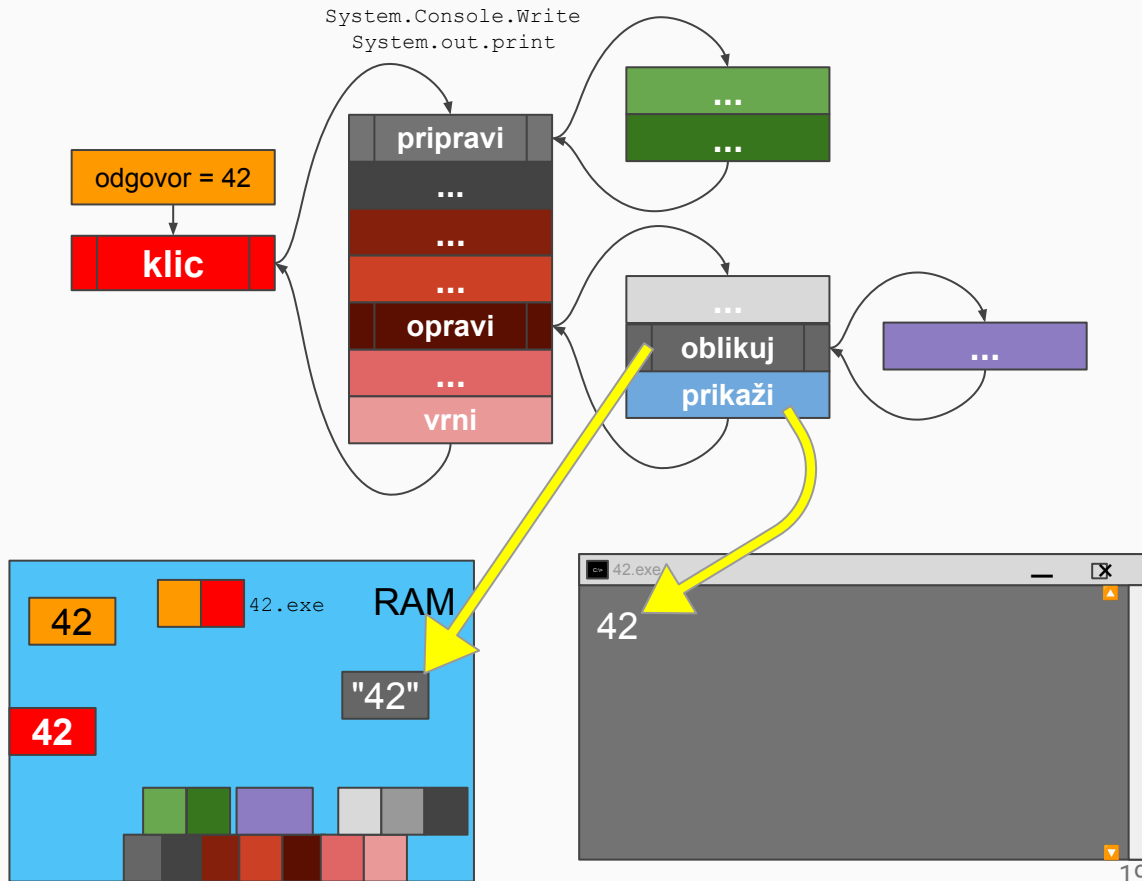
Vrednost, ki jo vrne funkcija, se klicajoči kodi posreduje na mestu klica funkcije.

Klici podprogramov so temelj programiranja in zato morajo imeti očiten namen. Dobra funkcija izpolni **eno** nalogo in **ne preseneti** klicatelja.

V ozadju...

Če se vrnemo na primer izpisa 42:

Pomnilnik vsebuje strojno kodo programa in vseh podprogramov; od podatkov je v pomnilniku "naša" 42, potem *argument** 42, ki smo ga posredovali podprogramu ob klicu in verjetno vsaj še *string* "42" (ali črki '4' in '2'), kar si bo naredil podprogram po svojih potrebah...

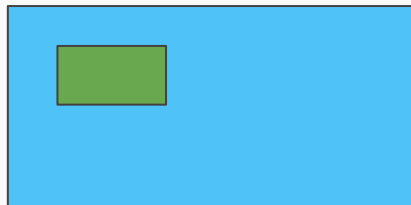


Deli-in-vladaj nad podatki ...

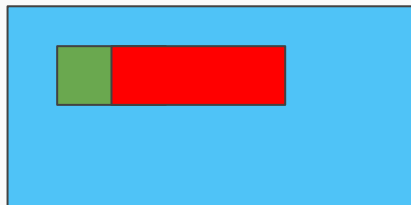
Program spremeni **velik** kos pomnilnika tako, da vsak izmed njegovih ukazov opravi **košček** te spremembe - funkcije **višjih nivojev** tipično opravijo največ sprememb, tiste iz **nizkih** pa manj. Tekom izvajanja programa lahko več različnih ukazov spreminja isti košček pomnilnika.

Višji ukaz je zgolj sekvenca nižjih ukazov, ki so spet sekvence še nižjih, dokler ne ostanejo le še "*primitivni*", procesorju razumljivi ukazi.

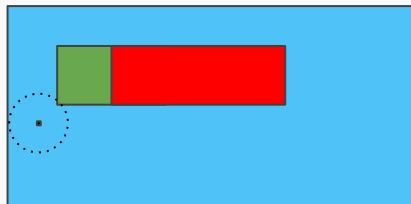
```
pridobivanje_podatkov(); // 2000 bytov
```



```
nek_izračun(); // 3000 bytov
```



```
odgovor = 42; // 4 byte
```



Naloga

Določi večje izmed dveh celih števil!

Obstaja več (veljavnih) rešitev te preproste naloge, tukaj bomo skušali poudariti nekaj pomembnih vidikov.

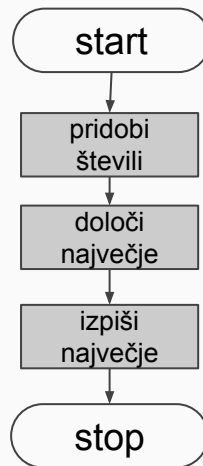
Pred programiranjem razčistimo osnove: Kaj hočemo, kaj vemo, kaj rabimo, zakaj delamo, kdo bo uporabljal, kaj bomo s programom, kako vemo, da ni napak...

Če želimo izbrati večjega izmed dveh števil lahko ustvarimo program, ki vpraša uporabnika po obeh številih in izpiše največjega. Poskusimo.

Program naj najprej pridobi obe števili, določi večjega in ga izpiše na ekran. *Deli in vladaj!*

Sedaj že vemo, da izpis opravimo s funkcijskim klicem, za vnos pričakujemo podobno.

Rešimo najprej vnos in izpis!



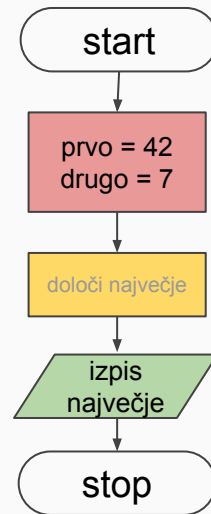
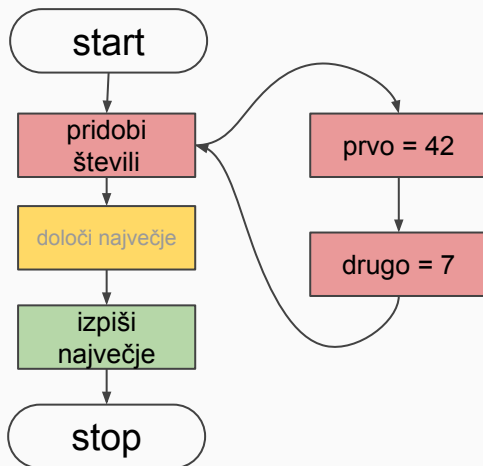
I/O

Branje števil iz tipkovnice je "čudno" v smislu, da ne moremo neposredno prebrati navadnega števila. Razlog je ta, da tipkovnica pošilja črke ('4', '2') in ne kar števila (42)... Vse rešitve zato vključujejo tip *string*.

Ker še ne poznamo stringov, bomo vnos nadomestili kar z 2 konstantama, recimo 42 in 7. Izpis že poznamo.

p.s.

To je super ideja, ker ob testiranju programa ne bomo rabili *znova in znova* tipkati števil 42 in 7!

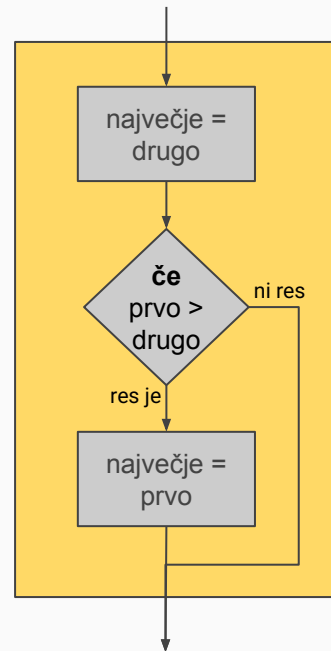
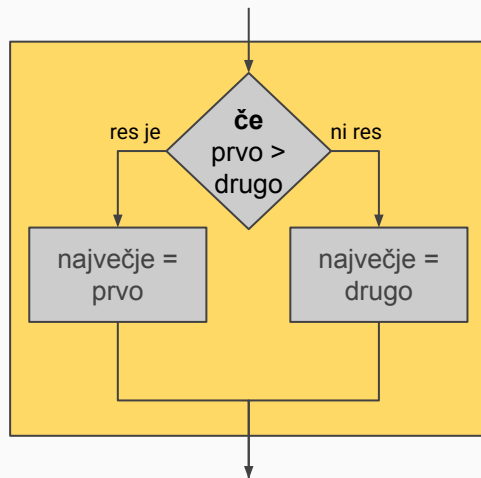
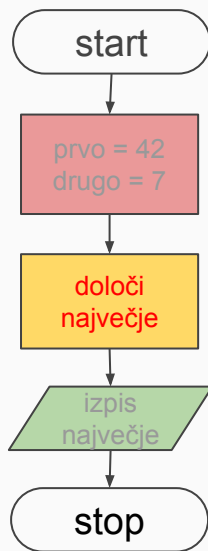


Določitev največjega števila

Algoritem bo očitno potreboval odločitev, ali je prvo število večje od drugega ali ne, odgovor pa moramo zapisati v spremenljivko *največje*, ki jo bere funkcija za izpis rezultata.

Za oranžni blok se ponujata dva glavna pristopa in recimo, da izberemo desni pristop.

Sedaj nas zanima, kakšen je dejanski program za izbran algoritem.



Program

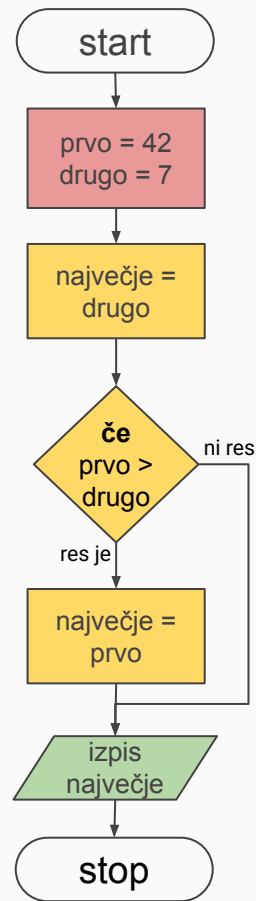
V telo glavne funkcije (*main*) sedaj napišimo ustrezno kodo. Prevajalnik takoj označi napako na neznani besedi (*final* | *const*). Popravimo in zaženemo. Odpre se novo okno in izpiše 42.

Smo končali? Poglejmo še nekaj različnih variant tega programa in njihove pluse in minuse...

```
static void Main(string[] args)
{
    // prva faza
    final int prvo = 42;
    const int drugo = 7;

    // druga faza
    int največje = drugo;
    if (prvo > drugo) {
        največje = prvo;
    }

    // tretja faza
    System.Console.Write(najvecje);
}
```



Najkrajše

Izpustimo lahko spremenljivko, ki je "shranila" vrednost večjega števila in kar znotraj odločitve izvedemo izpis.

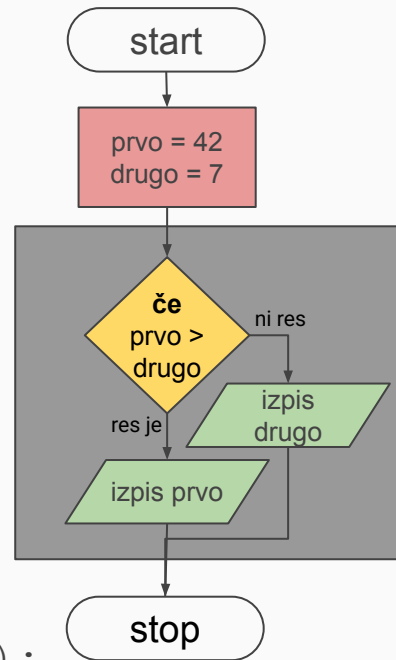
Rešitev je kratka in razumljiva in za ta šolski primer dobra. Njena glavna slabost je v tem, da je združila oranžno in zeleno fazo - odločanje in izpis - v en velik odločitveni blok.

Sivi blok opravi odločanje in izpis, dve nalogi! Bolje* je bilo prej, ko je oranžni blok opravil eno nalogo, zeleni pa drugo...

```
int prvo = 42;
```

```
int drugo = 7;
```

```
if (prvo > drugo) {  
    System.Console.Write(prvo);  
}  
else {  
    System.Console.Write(drugo);  
}
```



S funkcijo

Namesto združevanja oranžnega in zelenega bloka lahko poskusimo oranžni blok izvesti s funkcijo *vecji*.

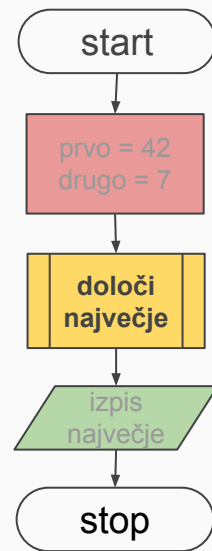
Definirati jo moramo pred* ali za funkcijo *main* in znotraj *class* bloka. Prednjo postavimo besedo *static* (do tega bomo še prišli v nadaljevanju).

Funkcija prejme kopijo števila 42 v **svojo** spremenljivko *prvo* ter kopijo števila 7 v svojo *drugo*. Njena *edina* naloga je določiti večje od teh dveh števil in ga vrniti klicatelju.

Glavni program nato izpiše rezultat...

```
static int vecje(int prvo, int drugo) {  
    int najvecje = drugo;  
    if (prvo > drugo)  
        najvecje = prvo;  
    return najvecje;  
}
```

```
static void main(String[] args)  
{  
    // priprava podatkov  
    final int prvo = 42, drugo = 7;  
    // obdelava  
    int najvecje = vecje(prvo, drugo);  
    // objava rezultatov  
    System.Console.Write(najvecje);  
}
```



Krajša funkcija

Podobno, kot smo odstranili spremenljivko *najvecje* v glavnem programu in združili oranžni in zeleni blok, lahko tudi funkcijo *vecje* izvedemo brez nje.

Ukaz *return* namreč takoj zaključi izvajanje funkcije in vrne rezultat.

Dejansko sta obe funkciji enakovredni, saj bo prevajalnik poskrbel za takšne in še večje optimizacije kode.

Obstaja pa še krajši zapis...

```
static int vecje(int prvo, int drugo) {  
    int najvecje = drugo;  
    if (prvo > drugo)  
        najvecje = prvo;  
    return najvecje;  
}
```

// krajša verzija

```
static int vecje(int prvo, int drugo) {  
    if (prvo > drugo)  
        return prvo;  
    return drugo;  
}
```

// najkrajša verzija

```
static int vecje(int prvo, int drugo) {  
    return prvo > drugo ? prvo : drugo;  
}
```

Izpis, test, ...

Namesto za izpis bi se glavni program lahko odločil tudi za test rezultata. Ker sta podatka 42 in 7 literala, posledično vemo, da mora biti rezultat 42.

In to preverimo s preprostim odločitvenim stavkom, ki izpiše opozorilo ob morebitni napaki.

Pravzaprav je takšno testiranje zelo pogosta naloga in je boljše od ročnega testiranja. Lahko bi napisali kar proceduro, ki opravlja takšen test. Poskusimo...

```
int prvo = 42;
int drugo = 7;

int najvecje = vecje(prvo, drugo);

if (42 != najvecje)
    System.Console.WriteLine("Napaka!?");

int tretje = 5;
int cetrtto = -3;

int najvecje2 = vecje(tretje, cetrtto);

if (5 != najvecje)
    System.Console.WriteLine("Napaka!?");
```

Test

Prva verzija sprejme število in ga primerja z 42... Hm, čudna je ta testna procedura - le kje vse rabimo proceduro za preverjanje, če je število enako 42?

Vidimo, da imamo hudo specializirano rešitev, ki je koristna samo na predavanjih.

Signal, da ni vse v redu, je že slabo ime procedure. Beseda *test* ne odraža stanja, je presplošna; to je v resnici procedura *opozori_ce_ni_42*.

```
static void test(int največje)
{
    if (42 != največje)
        System.Console.WriteLine("Napaka!?");
}
```

```
int prvo = 42;
int drugo = 7;

int največje = vecje(prvo, drugo);

test(največje);
test(33);           // OK, javi napako
```

Splošneje

Če preimenujemo še argument vidimo, da je naša testna procedura zelo zelo *specializirana*; poskusimo jo narediti bolj **splošno**.

Fino bi bilo, če ne bi preverjala pravilnosti samo za 42, ampak tudi za druge konstante. Ko kličemo naš test točno vemo, koliko je pričakovan rezultat, zato ga lahko posredujemo skupaj s testiranim številom.

Poskusimo.

```
static void opozori_ce_ni_42(int stevilo) {  
    if (42 != stevilo)  
        System.Console.WriteLine("Napaka!?");  
}  
  
static void opozori_ce_ni_enako(int stevilo,  
                                int pravilno)  
{  
    if (pravilno != stevilo)  
        System.Console.WriteLine("Napaka!?");  
}  
  
int najvecje = vecje(42, 7);  
opozori_ce_ni_enako(najvecje, 42);  
opozori_ce_ni_enako(vecje(5, -2), 5);
```

Univerzalna oblika

Že bolje. Ampak naš test zmore preverjati zgolj odstopanje od pravilne vrednosti... Kaj pa, če želimo v pogoju recimo manjše, večje?

Odločitveni stavek potrebuje zgolj logično vrednost - rezultat logičnega izraza. Slednjega lahko "ovrednotimo" že ob klicu testa!

S tem smo iz dveh argumentov prešli na enega in na funkcijo, ki preveri pravilnost podanega pogoja. Sedaj si zasluži (spet) novo ime...

Ker je

```
if (pravilno != stevilo) ...
```

pravzaprav okrajšava za

```
bool pogoj = pravilno == stevilo;  
if (!pogoj) ...
```

lahko *opozori_ce_ni_enako* preoblikujemo v *opozori_ce_ni_res* oziroma *preveri*:

```
void preveri (bool pogoj) {  
    if (!pogoj)  
        System.out.print ("Napaka!?" );  
}
```

```
preveri (vecje (42, 7) == 42);
```

Več testov na kupu

Sedaj lahko glavni program enostavno izvede več testnih scenarijev in tako temeljito preveri osrednjo funkcijo *vecje*. To poveča **varnost kode**, saj lahko vse morebitne spremembe hitro in *enostavno* stestiramo.

"Testiranje lahko pokaže samo prisotnost napak, nikdar pa ne dokaže njihove odsotnosti" (E. W. Dijkstra).

Uporabniški vmesnik (vpis in izpis števil) smo popolnoma zapostavili, ker bo o njem več govora drugod...

```
static void preveri(bool pogoj) {  
    if (!pogoj)  
        System.Console.WriteLine("Napaka!?");  
}  
  
static int vecje(int prvo, int drugo) {  
    if (prvo > drugo)  
        return prvo;  
    return drugo;  
}  
  
static void Main(string[] args) {  
    preveri( 42 == vecje(42, 7) );  
    preveri( 42 == vecje(7, 42) );  
    preveri( 42 == vecje(42, -7) );  
    preveri( 42 == vecje(-7, 42) );  
    preveri( -7 == vecje(-42, -7) );  
    preveri( -7 == vecje(-7, -42) );  
}
```


Operatorji

Dejansko bi lahko vse operatorje (+ - * / && ...) imeli v funkcijski obliki, npr:

```
int plus(int a, int b) {...}
```

vendar bi bili takšni programi daljši, manj intuitivni in zato slabši. Praktično vsi programski jeziki imajo zato vgrajene simbolne operatorje za osnovne matematične operacije.

Nekateri jeziki pa dovolijo programerju, da re-definira pomen izbranih simbolov za uporabo z njegovimi tipi podatkov (*operator overloading*):

```
otrok = oce + mama;
```

42 + 0

plus(42, 0)

6 * (3 + 2*2) + 0

((6 * (3 + (2*2))) + 0)

plus(krat(6, plus(3, krat(2,2))), 0)

Naloga

Program za izpis kvadrata z
zvezdicami z uporabo funkcij!?

...

Kvadrat - rešitev

Glavni program/funkcija ima eno nalogo - izris kvadrata. To nalogo lahko razbijemo na dve podnalogi:

- (1) pridobitev dimenzije, in
- (2) dejanski izris na konzolo.

Prvo podnalogo lahko nadomestimo z literalno konstanto ali klicem podprograma. Izris kvadrata bo tudi opravila temu namenjena funkcija *narisi_kvadrat_z_zvezdicami*.

Glavni program je torej lahko v vsaj dveh oblikah:

```
// A: izris kvadrata 4x4 z zvezdicami
public static void main(String[] args) {
    narisi_kvadrat_z_zvedicami(4);
}
```

```
// B: izris kvadrata dim x dim z zvezdicami
static void main(String[] args) {
    int dimenzija = beri_dimenzijo();
    narisi_kvadrat_z_zvedicami(dimenzija);
}

static int beri_dimenzijo()
{
    return 4;
}
```

Dodajanje funkcij

Dejansko smo uporabili proceduro *narisi_kvadrat_z_zvezdicami* še preden smo jo sploh definirali. Hkrati smo ji "povedali", da naj nariše kvadrat s štirimi zvezdicami kot stranico.

Sedaj moramo torej definirati manjkajočo funkcijo; spet deli-in-vladaj. Ker je računalniška konzola organizirana po vrsticah, je risanje kvadrata smiselno "razstaviti" na risanje zgornje vrstice, vmesnih vrstic in spodnje vrstice...

```
static void narisi_kvadrat_z_zvedicami
    (int dimenzija)
{
    narisi_zgornjo_vrstico_zvezdic(dimenzija);
    narisi_vmesne_vrstice_kvadrata(dimenzija);
    narisi_spodnjo_vrstico_zvezdic(dimenzija);
}
```

In še več funkcij...

Razmišljamo in delamo enako kot prej: razbijamo problem na manjše probleme in za vsakega predvidimo rešitev v obliki funkcije.

Nato definiramo manjkajoče funkcije, ki jih po potrebi spet razbijamo na manjše dele...

Funkciji za izris zgornje in spodnje vrstice sta identični, zato bi ju lahko nadomestili z eno funkcijo, recimo ji *izpis_polne_vrstice*, kar pa bi zametlo uporabljen razgradnjo problema in v toliko raje ohranimo ločeni funkciji...

```
static void narisi_zgornjo_vrstico_zvezdic
    (int dimenzija)
{
    izpisi_zvezdice(dimenzija);
    zakljuci_vrstico();
}

static void narisi_spodnjo_vrstico_zvezdic
    (int dimenzija)
{
    narisi_zgornjo_vrstico_zvezdic(dimenzija);
}

static void izpisi_zvezdice(int dolzina)
{
    for (int d=0; d < dolzina; ++d)
        izpisi_eno_zvezdico();
}
```

Osrednji del...

Osrednji del kvadrata lahko razbijemo na *dimenzija - 2* enakih vrstic, zato uporabimo zanko.

Vsako od vrstic lahko nadalje razbijemo na izris prve zvezdice, presledkov in zadnje zvezdice.

```
static void narisi_vmesne_vrstice_kvadrata
    (int sirina)
{
    int visina = sirina - 2;
    for(int v=0; v < visina; ++v) {
        narisi_vrstico_med_zvezdicama(sirina);
    }
}

static void narisi_vrstico_med_zvezdicama
    (int sirina)
{
    izpisi_eno_zvezdico();
    izpisi_presledke(sirina - 2);
    izpisi_eno_zvezdico();
    zakljuci_vrstico();
}
```

Delujoča rešitev

Prišli smo do točke, ko razbijanje na manjše probleme ni več potrebno, ker znamo vse probleme rešiti s pomočjo vgrajenih ukazov in sistemskih ter lastnih funkcij...

Opazimo pa lahko več pomanjkljivosti:

1. dolga opisna imena funkcij (zvezdice, presledki)
2. slaba ponovna uporaba funkcij
3. težko spreminjanje

```
static void izpisi_presledke(int dolzina)
{
    for (int d=0; d<dolzina; ++d)
        izpisi_en_presledek();
}

static void izpisi_eno_zvezdico()
{
    System.out.print('*');
}

static void izpisi_en_presledek()
{
    System.out.print(' ');
}

static void zakljuci_vrstico()
{
    System.out.println();
}
```

Izboljšave

Poskusimo narediti kodo bolj splošno.

Prva ideja je možnost, da bi lahko ob klicu krovne metode za izris določili, kateri znak naj se uporabi za črto; privzeto naj bo to zvezdica.

Izbira črke za rob je prepuščena klicatelju na najvišjem nivoju. Funkcije dobijo nov argument, ki ga po potrebi posredujejo naprej na še nižji nivo...

```
narisi_kvadrat(4, '#');
```

```
####
```

```
#  #
```

```
#  #
```

```
####
```

```
static void narisi_kvadrat
```

```
    (int dimenzija, char rob='*')
```

```
{
```

```
    narisi_zgornjo_vrstico(dimenzija, rob);
```

```
    narisi_vmesne_vrstice(dimenzija, rob);
```

```
    narisi_spodnjo_vrstico(dimenzija, rob);
```

```
}
```

```
itd...
```


Polnilo?

Če smo znali uvesti nov argument za rob kvadrata, lahko podobno uvedemo tudi argument za kvadratovo polnilo, da ne bo kvadrat vedno napolnjen zgolj s presledki...

To ima zanimivo posledico, da se funkciji *izpisi_en_presledek* in *izpisi_eno_zvezdico* sedaj **združita** v enotno obliko - *izpisi_en_znak*. Enako velja tudi za izpisa več zvezdic in presledkov:

```
narisi_kvadrat(4, '#', 'a');  
####  
#aa#  
#aa#  
####
```

```
static void izpisi_en_znak(char znak)  
{  
    System.out.print(znak);  
}  
  
static void izpisi_znake(int dolzina,  
                        char znak)  
{  
    for (int d=0; d<dolzina; ++d)  
        izpisi_en_znak(znak);  
}
```

Kvadrat?

Naša koda je še vedno specializirana za risanje kvadrata. Funkcije določajo število vrstic in njihovo dolžino na podlagi enega parametra - dimenzije.

Ampak splošnejša oblika kvadrata je pravokotnik. Naša koda bi bila bolj uporabna, če bi lahko posebej podali tako število vrstic kot tudi njihovo dolžino; izris kvadrata se potem zgodi vedno, ko sta obe dimenziji enaki...

```
static void narisi_kvadrat(int dimenzija,  
                           char rob, char polnilo)  
{  
    narisi_pravokotnik(dimenzija,  
                        dimenzija,  
                        rob,  
                        polnilo);  
}  
  
static void narisi_pravokotnik(int visina,  
                               int sirina, char rob, char polnilo)  
{  
    narisi_zgornjo_vrstico(sirina, rob);  
    narisi_vmesne_vrstice(visina-2, sirina, rob,  
                           polnilo);  
    narisi_spodnjo_vrstico(sirina, rob);  
}
```

Mejne vrednosti

Do sedaj se nismo vprašali, ali zmore naš program izrisati kvadrate različnih dimenzij.

Ker je vhodni argument *int* - celo število - se lahko zgodi, da je enak nič ali celo negativen.

Paziti moramo pri izrisu vmesnih in spodnje vrstice; rišemo jih samo v primeru pravokotnikov z višino večjo od ena.

```
static void narisi_pravokotnik(int visina,
                               int sirina, char rob, char polnilo)
{
    narisi_zgornjo_vrstico(sirina, rob);
    if (visina > 1) {
        narisi_vmesne_vrstice(
            visina-2, sirina, rob, polnilo);
        narisi_spodnjo_vrstico(sirina, rob);
    }
}

*      **      ***   itd...
      **      *  *
           ***
```

Zaključek

Predstavljen postopek je resda *overkill* za risanje kvadratov, ki je kot enostaven problem rešljiv tudi brez toliko funkcij. V realni kodi recimo nikoli ne bomo srečali procedure *izpisi_en_presledek()*!

Vendar je predstavljen princip UNIVERZALEN in v toliko NUJEN pri programiranju zahtevnih rešitev, ki jih je praktično NEMOGOČE ustvariti kako drugače. Na dolgi rok lahko velik program obvladujemo samo z njegovo razgradnjo na manjše gradnike.

Funkcije so zakon!

(če jih uporabimo pametno)